

Comparing Two C Database Prototypes for Large Publishing Networks

Simant Bhattarai | KIT205 Assessment Task 1 | https://github.com/Simantb/KIT205/tree/main/assessment1_database



Verified package status

Both prototypes compiled and ran successfully on 18 May 2026. Unit tests: 14 passed, 0 failed.

Introduction

This project prototypes a database-style system for a publishing catalogue. The data model is many-to-many: one **book** can have many **authors**, and one author can contribute to many books. Large digital catalogues need this relationship to stay correct, ordered, and fast at scale.

Required Operations

- Insert a book-author relationship.
- Print an ordered list of all books.
- Print an ordered list of all authors for a given book.
- Print an ordered list of all books for a given author.

Methodology

The program generates simulated book-author relationships at five scales: 1,000, 3,000, 6,000, 12,000, and 24,000 relationships. Each prototype processes the same generated data. Two time-critical operations are measured: total insertion time and repeated author-lookups by book ID.

Conclusion

Prototype 2 is the stronger recommendation for very large datasets in this domain. Its hash-table record lookup keeps both insertion and query time substantially lower than Prototype 1 as the database grows. Prototype 1 still provides a correct ordered tree-based design, but linked-list record search becomes the dominant bottleneck.

Prototype 1

- Sorted linked list of books
- Sorted linked list of authors
- AVL tree of authors per book
- AVL tree of books per author

Prototype 2

- Hash table of books
- Hash table of authors
- Sorted dynamic array per book
- Sorted dynamic array per author

Why These Structures?

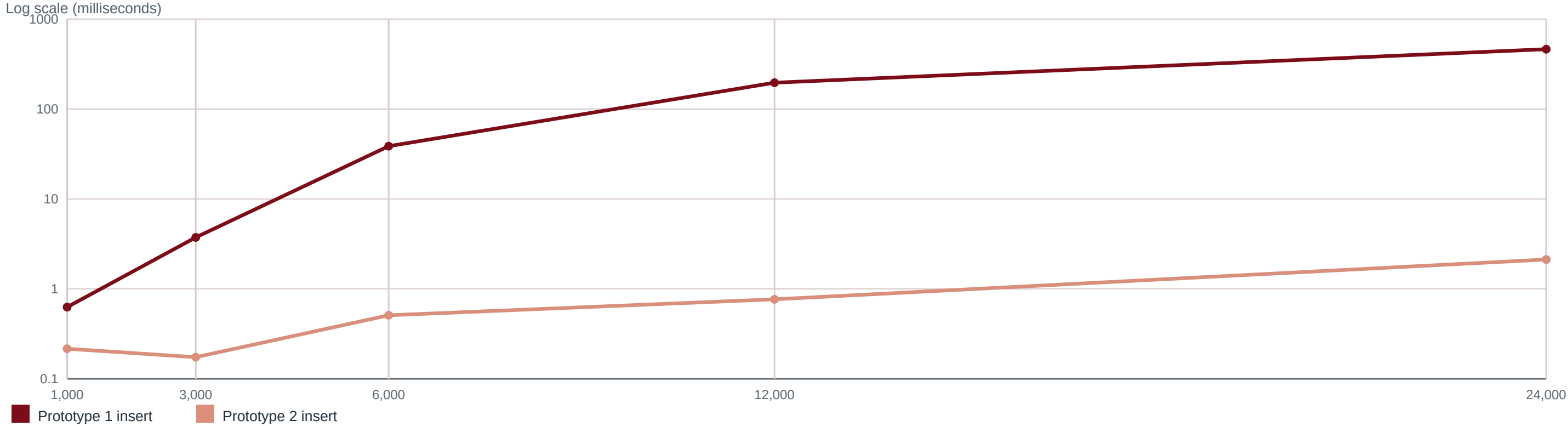
Prototype 1 combines linked lists and balanced trees to preserve order naturally, but record discovery still requires linear traversal. Prototype 2 trades tree-based ordering for average-case constant-time hash lookup, while using sorted arrays to preserve ordered output.

Operation	Prototype 1	Prototype 2
Insert relationship	$O(B + A + \log r)$	$O(1 \text{ avg. lookup} + r \text{ shift})$
List all books	$O(B)$	$O(B \log B)$
Authors for a book	$O(B + r)$	$O(1 \text{ avg.} + r)$
Books for an author	$O(A + r)$	$O(1 \text{ avg.} + r)$

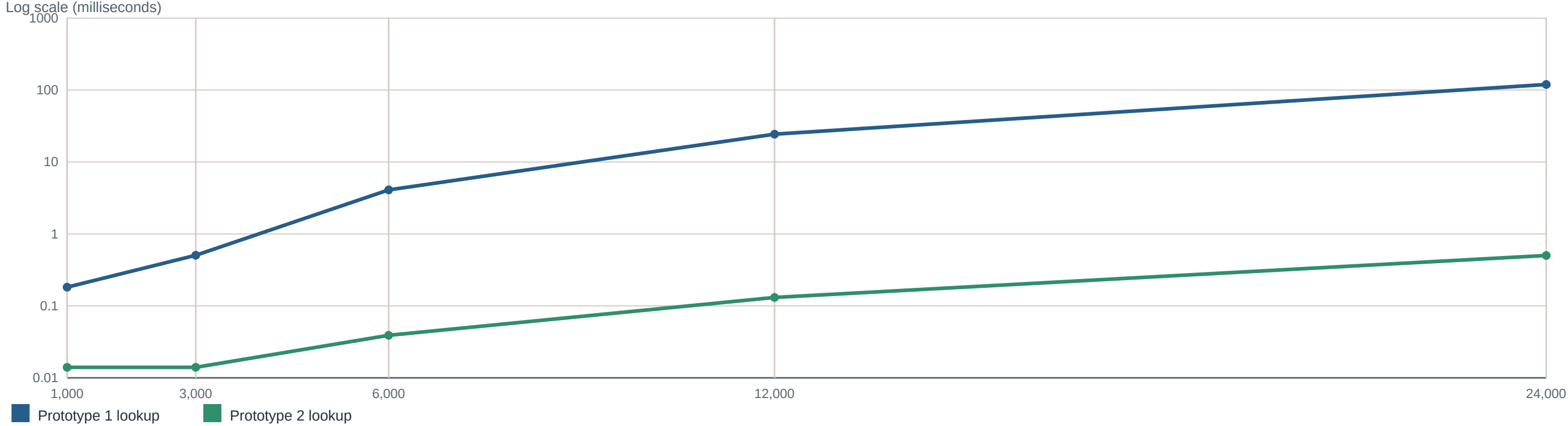
References

KIT205 lecture and tutorial materials; C standard library documentation; algorithm notes for AVL trees and hash tables; GitHub repository for version control; AI support used for drafting and implementation assistance.

Insertion Performance



Lookup Performance



Relationships	P1 Insert	P2 Insert	P1 Lookup	P2 Lookup
1,000	0.627	0.216	0.182	0.014
3,000	3.737	0.174	0.505	0.014
6,000	38.636	0.509	4.094	0.039
12,000	196.220	0.766	24.358	0.131
24,000	462.762	2.122	119.528	0.502